abstraction can lead to problematic database query patterns in your apps. A classic example is the *N*+1 problem, in which what should be a single database request turns into separate requests for every single row in a database table.

Another commonly cited drawback is performance. ORMs are abstractions over several concepts, so they inherently do more work than if you were to handcraft every piece of data access in your app. Most ORMs, EF Core included, trade some degree of performance for ease of development.

That said, if you're aware of the pitfalls of ORMs, you can often drastically simplify the code required to interact with a database. As with anything, if the abstraction works for you, use it; otherwise, don't. If you have only minimal database access requirements or need the best performance you can get, an ORM such as EF Core may not be the right fit.

An alternative is to get the best of both worlds: use an ORM for the quick development of the bulk of your application, and fall back to lower-level APIs such as ADO.NET for those few areas that prove to be bottlenecks. That way, you can get good-enough performance with EF Core, trading performance for development time, and optimize only those areas that need it.

NOTE These days, the performance aspect is one of the weaker arguments against ORMs. EF Core uses many database tricks and crafts clean SQL queries, so unless you're a database expert, you may find that it outperforms even your handcrafted ADO.NET queries!

Even if you decide to use an ORM in your app, many ORMs are available for .NET, of which EF Core is one. Whether EF Core is right for you depends on the features you need and the tradeoffs you're willing to make to get them. Section 12.1.3 compares EF Core with Microsoft's other offering,